영남대학교 자동차소프트웨어학과 · 3-2 전공선택

자동차센서

1강 — 우리가 만들 것:
자율주행 자동차, 임베디드 시스템, 개발 환경

프로젝트 기반 과정 · 수강 2명 · TOPST D3-G + PiRacer

카메라 초음파 레이더 라이다 GNSS IMU

LECTURE 01 / 15

강의 90분 + 실습 90분

오늘 90분

강의 90분 뒤에 실습 90분이 이어진다. 실습에서는 개발 PC를 세운다.

시간	파트	주제	핵심 메시지
00–10	A	**우리가 만들 것**	학기 말에 이 차가 이렇게 굴러간다
10–35	B	**자율주행 자동차 개념**	자율주행은 인지–판단–제어 루프이고, 인지의 시작은 센서다
35–60	C	**임베디드 시스템과 차량 컴퓨팅**	자동차는 바퀴 달린 컴퓨터 네트워크이고, 우리 보드는 그 축소판이다
60–80	D	**개발 환경**	PC에서 짜서 보드에서 돌린다
80–90	E	**정리 · 다음 주**	다음 주엔 보드를 켠다
+90	실습	**개발 PC 셋업**	Ubuntu 22.04 · GitHub 랩노트 저장소 · SSH 키 · 역할 논의

중간에 질문을 세 번 던진다. 2명이므로 두 사람 모두 답한다. 정답을 맞히는 자리가 아니라 근거를 말하는 자리다.

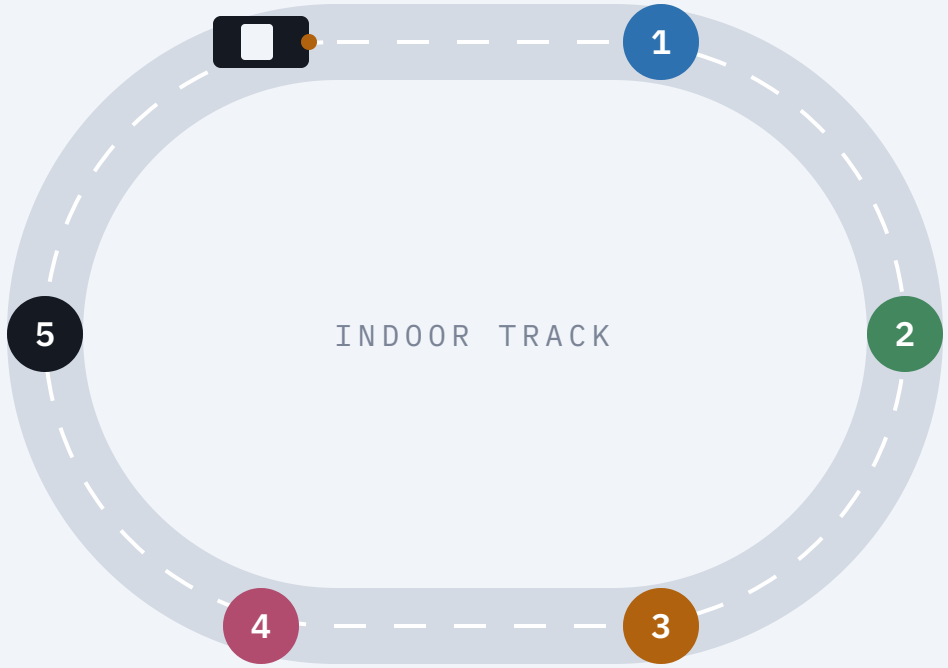
A

우리가 만들 것

> 학기 말에 이 차가 이렇게 굴러간다.

15주차, 실내 트랙에서 이것을 시연한다

다섯 가지 시나리오. 각 시나리오는 서로 다른 센서가 담당한다.



01 카메라 차선 인식 자율 주행
트랙 1바퀴 완주 · MIPI-CSI 카메라 → 차선 검출 → 조향

02 전방 장애물 감지 후 정지 · 회피
초음파 + 2D 라이다 · 긴급제동(AEB)의 축소판

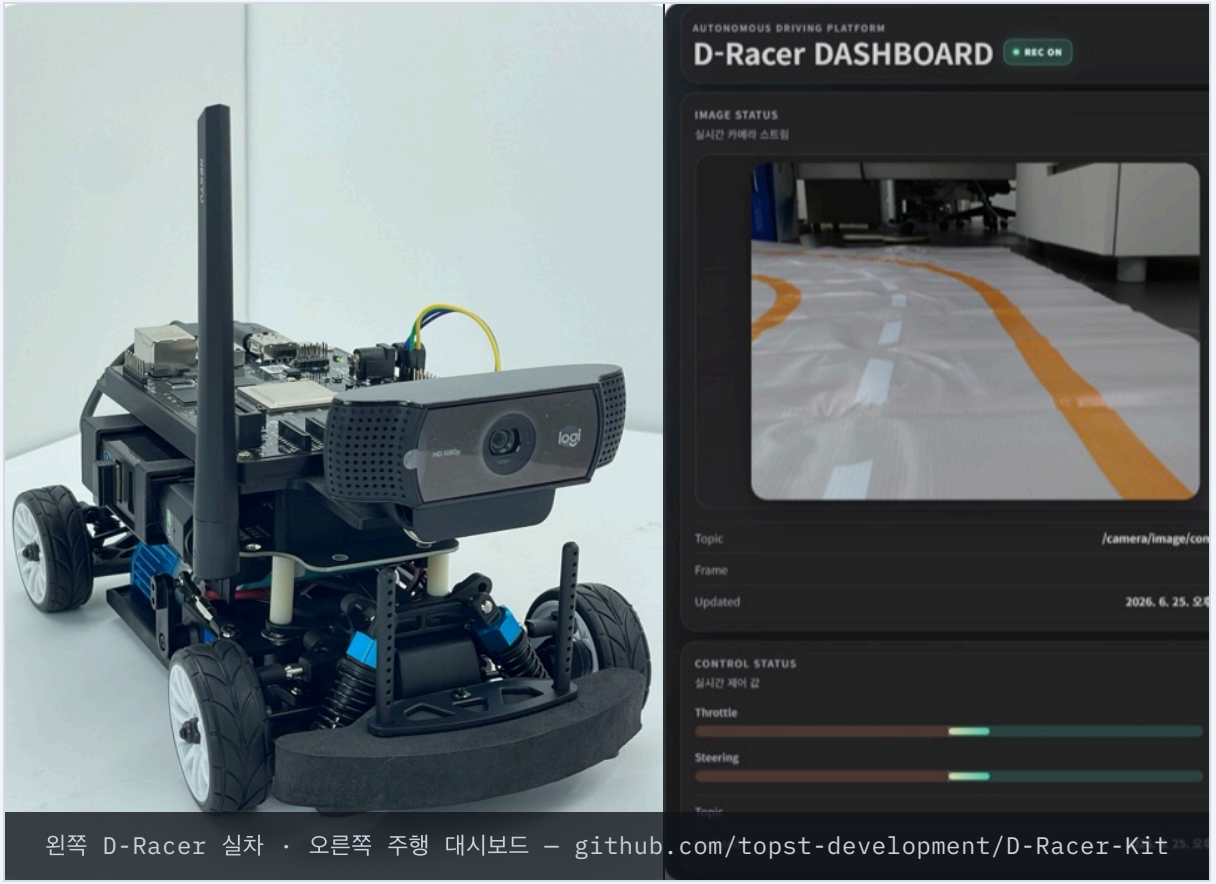
03 접근 물체 속도 측정
HB100 도플러 레이더 모듈 · 상대속도 산출

04 주행 궤적 로깅
GNSS + IMU · 주행 후 궤적 시각화

05 6종 센서 데이터 실시간 통합 표시
ROS2 토픽 6개 동시 발행 + 대시보드

플랫폼: D-Racer — TOPST D3-G + PiRacer 섀시

1/10 스케일 차량 1대를 두 사람이 공동으로 구축한다.



왼쪽 D-Racer 실차 · 오른쪽 주행 대시보드 — github.com/topst-development/D-Racer-Kit

컴퓨팅	TOPST D3-G — 텔레칩스 TCC8050 기반 차량용 SoC 보드
섀시	PiRacer — 1/10 스케일, DC 구동 모터 + 조향 서보
OS	Ubuntu 22.04
미들웨어	ROS2 Humble — 센서마다 노드 1개, 토픽으로 연결
센서	MIPI-CSI 카메라 · 초음파 · 2D 라이다 · 도플러 레이더 · GNSS · IMU

> 이 플랫폼은 **2026 SEA:ME 해커톤**(폭스바겐그룹코리아 후원, 국민대 주관)의 공식 플랫폼이다. 학기 결과물을 그대로 대회나 캡스톤디자인으로 이어갈 수 있다.

국산 차량용 반도체 위에서 만든다. 왜 라즈베리파이가 아니라 이 보드인지는 파트 C에서 다룬다.

2명, 차 한 대 — 공동 플랫폼 + 개인 담당

담당은 프로젝트에서 책임지는 범위이지 배우는 범위가 아니다. 이론과 중간시험은 6종 전부.

학생 A

인지 · 비전 (Perception / Vision)

MIPI-CSI 카메라, USB 웹캠 — 영상 파이프라인, 캘리브레이션, 차선 검출, 조향각 산출

`/camera/image_raw` · `/lane/steering_angle`

학생 B

측거 · 측위 (Ranging / Localization)

초음파, 도플러 레이더, 2D 라이다, GNSS, IMU — 거리·속도 측정, 장애물 검출, 궤적 추정

`/ultrasonic/range` · `/scan` · `/radar/speed` · `/gps/fix` · `/imu/data`

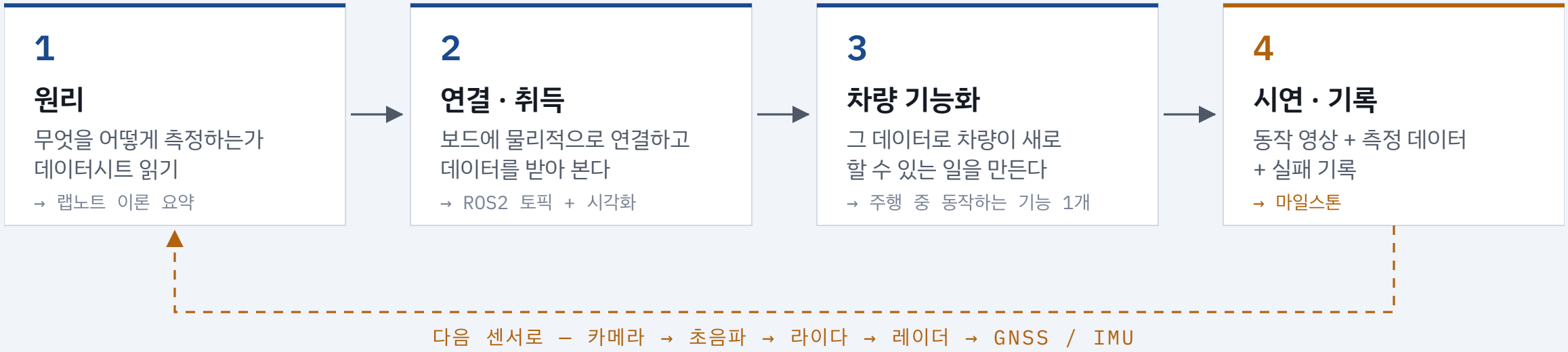
공통 담당: 시스템 아키텍처, 차량 제어(모터·서보) 노드, 통합, 대시보드, 최종 시연. 매주 랩미팅에서 자기 담당을 상대에게 설명하고, 8강·14강에 **상대 담당 센서 구술 확인**을 받는다.

프로젝트	주간 랩노트	중간 점검	참여 · 상호교육
50%	20%	20%	10%

마일스톤은 성공/실패로 채점하지 않는다. 실패한 시도라도 랩노트에 원인 분석과 다음 가설이 남아 있으면 만점이 가능하다. 하드웨어 프로젝트는 원래 절반이 실패한다. 감추지 말고 기록할 것.

센서를 하나 얹을 때마다 같은 4단계를 반복한다

이 4단계가 이 과정의 학습 단위다. 카메라 · 초음파 · 라이다 · 레이더 · GNSS/IMU에 대해 반복된다.



15회차 — 차를 먼저 굴리고, 센서를 하나씩 얹는다

회차당 강의 90분 + 실습 90분. 실제 개발의 대부분은 주중 자율 작업이다.



B

자율주행 자동차 개념

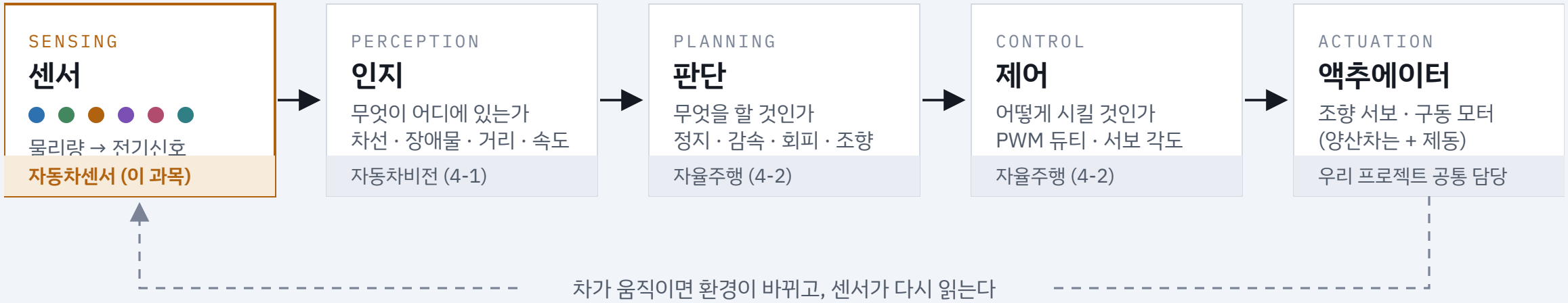
> 자율주행은 인지–판단–제어 루프이고,
> 인지의 시작은 센서다.

자율주행은 하나의 루프다 — 인지 · 판단 · 제어

이 루프가 초당 수십 번 돈다. 어느 한 칸이라도 늦으면 차는 늦게 반응한다.

사람의 운전과 같은 순서다 — **보고 · 판단하고 · 손발을 움직인다.** 자율주행은 이 셋을 기계가 한다.

아래 회색 칸은 후속 과목이 맡는다. 우리는 맨 왼쪽 칸 — 데이터가 소프트웨어에 도달하는 지점까지를 책임진다.



센서는 "장애물"을 주지 않는다 — 물리량을 준다

해석은 전부 소프트웨어의 몫이다. 그래서 무엇을 측정하는지 아는 것이 먼저다.

센서	측정하는 물리량	매개 물질 · 파장	소프트웨어가 받는 것
카메라	가시광의 세기와 색	전자기파 · 400–700 nm	2차원 화소 배열 (프레임)
초음파	음파의 왕복 시간	음파 · 약 40 kHz	거리값 1개
레이더	전파의 반사 지연 · 주파수 변화	전자기파 · 24 / 77 GHz	거리 · 상대속도 · 각도
라이다	레이저 펄스의 왕복 시간	전자기파 · 근적외선	3차원 점군 (point cloud)
GNSS	위성 신호의 도달 시간차	전자기파 · L 밴드	위도 · 경도 · 고도
IMU	가속도와 각속도	관성 (역학)	자세 · 상대 이동량

어떤 센서도 "앞에 사람이 있다"고 말해 주지 않는다. **시간 · 세기 · 주파수**를 줄 뿐이다. 그 숫자를 의미로 바꾸는 것이 인지(Perception)이고, 그 전에 **숫자를 정확히 받아오는 것**이 이 과목이다.

자동화 레벨 0–5 — 경계는 L2와 L3 사이다

SAE J3016 기준. 레벨을 가르는 것은 성능이 아니라 **누가 주행 환경을 감시하는가**이다.



우리 차는 어디인가. 정해진 실내 트랙, 정해진 조도. 조건이 극단적으로 좁아 레벨을 붙이는 것이 무의미하다. 우리가 만드는 것은 **L1–L2 기능(LKA · AEB)의 축소 재현**이지 자율주행차가 아니다.

기능이 센서를 정한다

양산차의 ADAS 기능마다 쓰는 센서가 다르다. 우리 프로젝트는 그 축소판을 하나씩 만든다.

ADAS 기능	하는 일	주 센서	우리 프로젝트에서 대응하는 것
ACC 적응형 순항	앞차와 간격을 유지하며 정속 주행	● 전방 레이더 (+ 카메라)	도플러 레이더로 접근 속도 측정 **10–12강**
AEB 긴급 제동	충돌이 임박하면 스스로 제동	● 레이더 + ● 카메라	초음파 정지 · 라이다 감속/회피 **6 · 9강**
LKA 차선 유지	차선을 벗어나지 않게 조향을 보조	● 전방 카메라	차선 검출 → 조향각 산출 **5–6강**
BSD 사각지대	옆·뒤에서 접근하는 차를 알린다	● 후측방 코너 레이더	개념만 다룬다 **11강**
PAS 주차 보조	근거리 장애물까지의 거리를 알린다	● 초음파 (범퍼 다수)	HC-SR04 전방 감지 후 정지 **6강**
측위 · 경로	차가 지금 어디에 있는가	● GNSS + ● IMU	주행 궤적 로깅 후 시각화 **12강**

패턴이 보인다 — **무엇인지 알아야 하면 카메라, 얼마나 멀고 빠른지 알아야 하면 레이더, 아주 가까우면 초음파.** 하나의 기능에 두 센서가 붙는 경우(AEB)는 한 센서의 실패를 다른 센서가 덮기 위해서다.

같은 문제, 다른 답 — 회사마다 센서 구성이 다르다

정답이 하나였다면 구성도 하나였을 것이다. 각자 다른 것을 타협했다.

CASE A

카메라 중심 — 테슬라

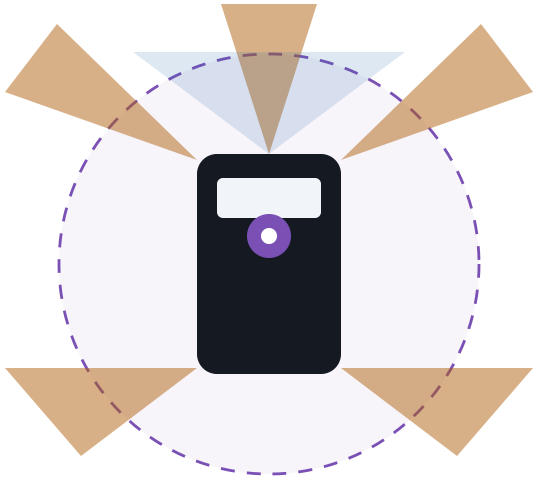


● 카메라 다수 (전방 · 측방 · 후방)

사람이 눈으로 하는 일을 카메라로 한다.
원가가 낮고 정보량이 크지만, 저조도 · 악천후 · 거리 추정에서 약해진다.

CASE B

라이다 포함 다중 — 웨이모

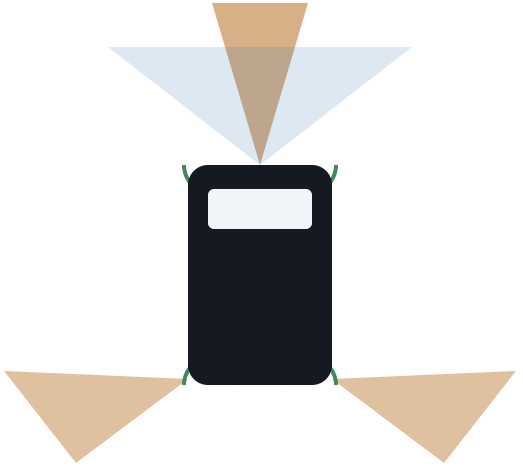


● 라이다 ● 레이더 ● 카메라

서로 다른 물리 원리를 겹쳐 놓는다.
한 종류가 실패해도 나머지가 남지만, 원가 · 계산량 · 정렬 부담이 커진다.

CASE C

양산차 ADAS — 현대 등



● 카메라 ● 레이더 ● 초음파

기능마다 필요한 최소 센서만 붙인다.
전방 카메라 + 레이더, 근거리는 초음파.
원가와 성능의 타협점이 여기다.

모든 항목에서 이기는 센서는 없다

이 표가 이 과목의 결론이자 센서퓨전의 출발점이다.

	야간 저조도	악천후 비 · 안개 · 눈	검출 거리 최대	공간 해상도 형상 파악	속도 직접 측정	색 · 문자 식별	상대 가격 많을수록 비쌈
카메라 가시광	● ○ ○	● ○ ○	● ● ○	● ● ●	● ○ ○	● ● ●	● ○ ○
초음파 음파	● ● ●	● ● ○	● ○ ○	● ○ ○	● ○ ○	○ ○ ○	● ○ ○
레이더 전파	● ● ●	● ● ●	● ● ●	● ○ ○	● ● ●	○ ○ ○	● ● ○
라이다 레이저	● ● ●	● ● ○	● ● ●	● ● ●	● ○ ○	○ ○ ○	● ● ●

채워진 원이 많을수록 그 항목에서 강하다 (3단계 상대 비교). GNSS · IMU는 환경을 보지 않는다 — 위치와 자세만 준다. 하늘이 가리면 GNSS는 끊기고, IMU는 시간이 지날수록 표류(drift)한다.

QUESTION 01

안개 낀 밤, 어떤 센서가 살아남나?

두 사람 모두 답한다. 답보다 **근거**가 중요하다 — 그 센서가 쓰는 매개 물질이 안개와 어둠에 어떻게 반응하는가로 설명할 것.
카메라 · 초음파 · 레이더 · 라이다 · GNSS · IMU 중에서 고른다. 이 질문에 대한 본인의 답은 **랩노트 #1**에 적어 온다.

카메라 초음파 레이더 라이다 GNSS IMU

그래서 여러 센서를 쓴다

한 센서를 더 좋게 만드는 것보다, 성격이 다른 센서를 겹치는 편이 확실하다.

카메라

무엇인가

차선, 신호등 색, 표지판 글자, 보행자인지 자전거인지 — 의미를 읽는다.

약점: 밤 · 역광 · 안개, 거리 추정이 부정확

레이더

얼마나 멀고 빠른가

거리와 상대속도를 한 번에, 그것도 비·안개·어둠에 거의 영향 없이 얻는다.

약점: 형상을 못 본다. 무엇인지 모른다

라이다

어디에 얼마나 정확히

3차원 형상과 거리를 센티미터 단위로. 지도 작성과 정밀 측위의 기준이 된다.

약점: 악천후에 약하고 비싸다

서로의 빈칸을 메우는 것이 센서퓨전(Sensor Fusion)이다. 그리고 겹치는 이유는 성능만이 아니다 — 한 센서가 **틀린 답을 확신에 차서 내놓을 때** 그것을 알아채려면 다른 원리의 센서가 필요하다. 이것이 중복(redundancy)이다.

이 과목에서는 13강에 퓨전을 개관만 한다. 본격적인 융합 이론은 **자율주행(4-2)**의 몫이다. 우리의 목표는 그 전 단계 — 여섯 종류의 데이터를 **정확하고 동시에** 받아 오는 것이다.

C

임베디드 시스템과 차량 컴퓨팅

> 자동차는 바퀴 달린 컴퓨터 네트워크이고,
> 우리 보드는 그 축소판이다.

임베디드 시스템 — 정해진 한 가지를 정해진 시각에

여러분이 쓰는 노트북과 무엇이 다른가. 성능이 아니라 **전제**가 다르다.

	범용 PC	임베디드 시스템
목적	무엇이든 — 사용자가 정한다	정해진 한 가지 — 설계자가 정한다
시간	평균 성능. 빠를수록 좋다	**마감 시각.** 늦은 정답은 오답이다
자원	넉넉하다. 부족하면 늘린다	전력 · 메모리 · 발열 · 원가가 모두 고정
환경	실내 상온, 안정된 전원	혹한 · 혹서 · 진동 · 전원 변동
고장했을 때	재부팅한다	사람이 다칠 수 있다
예	노트북, 서버	에어백 ECU, ABS, 카메라 모듈, **우리 D3-G**

> **실시간(real-time)은 "빠르다"가 아니다.** 정해진 시각까지 **반드시** 끝난다는 보장이다. 평균 1 ms에 끝나지만 가끔 200 ms가 걸리는 시스템보다, 항상 10 ms 안에 끝나는 시스템이 자동차에서는 낫다. 자동차에서 중요한 것은 평균이 아니라 **최악의 경우**다.

세 종류의 컴퓨터 — 크기가 아니라 성격이 다르다

2학년 MCU프로그래밍에서 만진 것이 왼쪽 열이다. 우리는 오른쪽 열을 쓴다.

	MCU — 아두이노 UNO	MPU / SOC — 라즈베리파이 4 B	차량용 SOC — TOPST D3-G
프로세서	ATmega328P 8비트 AVR · 16 MHz · 1코어	Broadcom BCM2711 Cortex-A72 ×4 · 1.8 GHz	텔레칩스 TCC8050 (Dolphin3) Cortex-A72 ×4 1.69 GHz · A53 ×4 1.45 GHz · **R5 600 MHz**
메모리	SRAM 2 KB · Flash 32 KB	LPDDR4 1–8 GB	LPDDR4X 4 GB 또는 8 GB · eMMC 32 GB
소프트웨어	OS 없음 (베어메탈)	리눅스	리눅스(A72·A53) + FreeRTOS(R5)
시간 특성	**결정적** — µs 단위로 예측된다	**비결정적** — 스케줄러에 맡긴다	**둘 다** — 코어를 나눠 갖는다
잘하는 일	핀 하나를 정확한 타이밍에 흔들기	영상 · 네트워크 · 파일, 풍부한 생태계	영상 처리와 실시간 제어를 한 보드에서

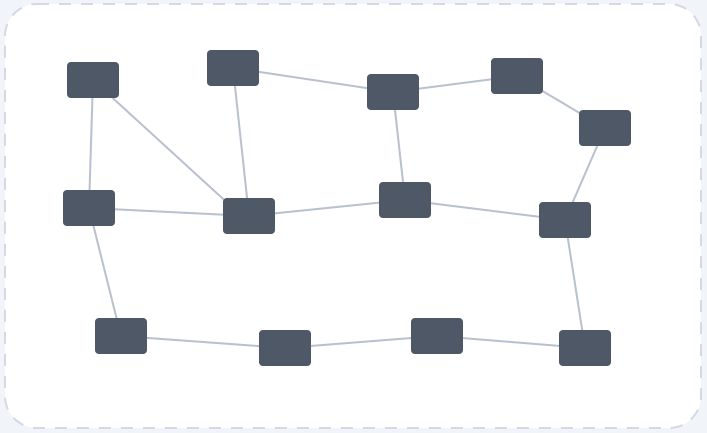
출처: topst.ai/product/g/d3 · github.com/topst-development/Documentation (D3-G Specification) · raspberrypi.com 제품 사양

자동차 안의 컴퓨터 구조는 지금 다시 그려지고 있다

기능마다 컴퓨터를 하나씩 붙이던 방식이 배선과 무게의 한계에 부딪혔다.

1980s – 2010s

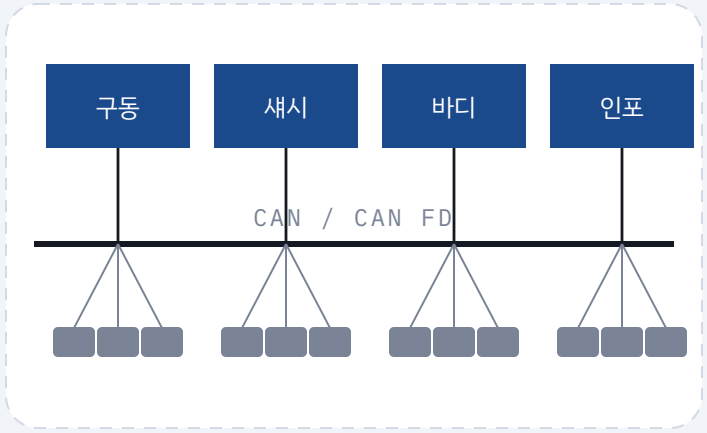
분산 (Distributed)



기능 하나에 ECU 하나. 고급차는 100개를 넘는다.
문제는 성능이 아니라 **배선** — 무게 · 원가 ·
고장 진단 · 소프트웨어 업데이트가 모두 막힌다.

2015 –

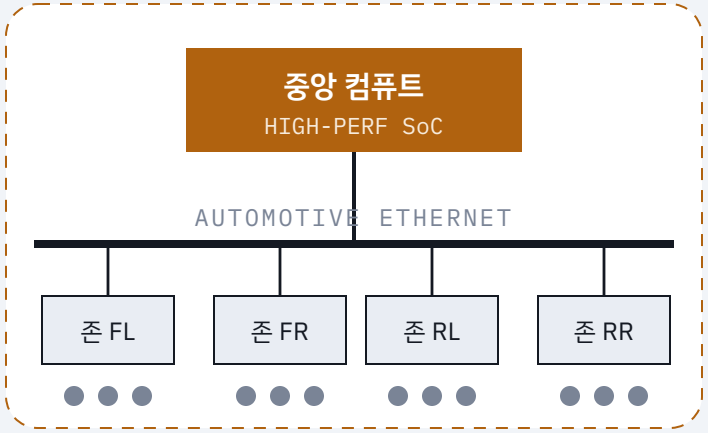
도메인 (Domain)



비슷한 기능을 묶어 도메인 컨트롤러 하나에
몰아 넣는다. 작은 ECU들은 그 아래로 내려간다.
배선은 줄었지만 도메인 경계를 넘는 기능이 늘었다.

2020 – 현재 진행형

존 · 중앙집중 (Zonal)

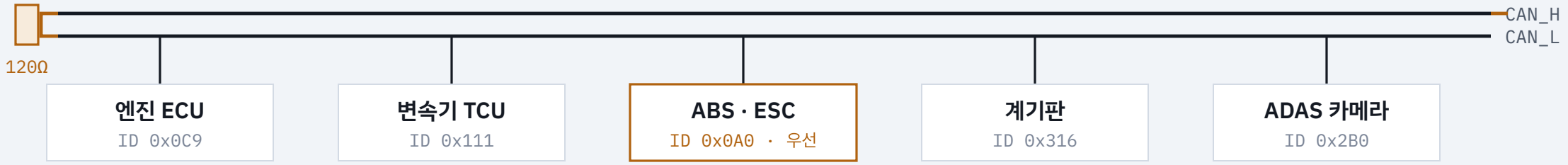


배선은 **위치(존)** 기준으로, 연산은 중앙으로.
센서는 가까운 존에 꽂고 데이터만 이더넷으로
보낸다. 차량이 하나의 컴퓨터에 가까워진다.

ECU들은 CAN 한 쌍의 선으로 대화한다

주소가 없다. 모두가 모두의 말을 듣고, 필요한 것만 골라 쓴다.

두 가닥의 **전압 차이**로 0과 1을 보낸다. 차동 신호라서 엔진 옆의 전기 잡음에도 버틴다. 양 끝은 120 Ω으로 종단한다.



CAN FRAME



ID가 곧 우선순위다. 두 노드가 동시에 말하면 ID 값이 작은 쪽이 버스를 가져가고, 진 쪽은 조용히 물러났다가 다시 보낸다. 그래서 브레이크가 라디오를 이긴다.

> Classical CAN은 최대 **1 Mbit/s**, 데이터 **8바이트**다. 카메라 영상은 여기에 실을 수 없다 — 그래서 차량 이더넷이 등장했다. **D3-G는 CAN 3채널을 제공한다**(트랜시버는 별도 보조 보드). 우리 프로젝트에서는 직접 쓰지 않지만, 차량 통신의 기준선으로 알아 둔다.

우리 보드 — TOPST D3-G

국산 차량용 SoC를 얹은 단일 보드 컴퓨터. 크기는 120 × 90 mm, 전원은 DC 5 V / 5 A.

컴퓨팅

SoC	텔레칩스 **TCC8050** (Dolphin3 계열)
CPU	Cortex-A72 ×4 @1.69 GHz Cortex-A53 ×4 @1.45 GHz Cortex-R5 @600 MHz
GPU	PowerVR 9XTP GT9524 · 168 GFLOPS
메모리	LPDDR4X **4 GB / 8 GB** (32비트 ×2채널)
저장	eMMC 5.1 32 GB (64 GB 옵션) + microSD
OS	Ubuntu 22.04 또는 Yocto(Kirkstone) · 커널 5.10

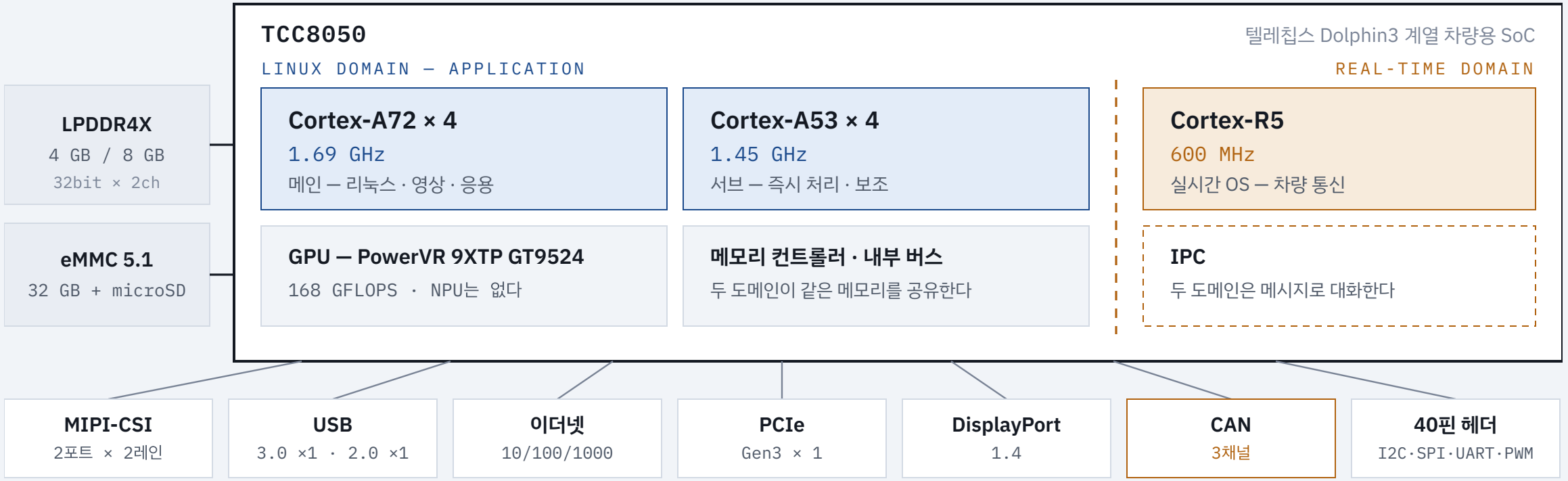
인터페이스

카메라	**MIPI-CSI 2포트 × 2레인** (4레인은 20핀 커넥터 옵션)
확장 헤더	**40핀** (2×20, 2.54 mm) — 라즈베리파이 HAT 호환 I2C · SPI · UART · I2S · PWM · GPIO
차량 통신	CAN **3채널** (트랜시버는 별도 보조 보드)
네트워크	이더넷 10 / 100 / 1000 Mbps ×1 · 무선은 USB 동글
USB	3.0 Type-A ×1 · 2.0 Type-A ×1 · Type-C ×1(펌웨어 전용)
기타	DisplayPort 1.4 (HDMI는 변환기 필요) · PCIe Gen3 ×1

출처: topst.ai/product/g/d3 · github.com/topst-development/Documentation → Single Board Computer / G Model / D3-G / Hardware / 1. Specification.md

한 칩 안에 성격이 다른 코어가 섞여 있다

리눅스가 도는 응용 코어와, 리눅스를 쓰지 않는 실시간 코어가 같은 칩에 들어 있다.



왜 성격이 다른 코어를 한 칩에 섞나

리눅스는 편하지만 **언제 끝날지 약속하지 못한다.** 그런데 차에는 약속이 필요한 일이 있다.

응용 코어 — CORTEX-A72 / A53

리눅스가 돈다

- 파일시스템 · 네트워크 · 드라이버 · 라이브러리가 전부 갖춰져 있다
- OpenCV, GStreamer, ROS2를 그대로 쓴다
- 대신 스케줄러가 언제 내 코드를 실행할지 **보장하지 않는다** — 가끔 수십 ms가 밀린다

잘 맞는 일: 영상 처리 · 경로 판단 · 로깅 · 통신

실시간 코어 — CORTEX-R5

실시간 OS(FreeRTOS)가 돈다

- OS가 작고 단순해서 응답 시간이 예측 가능하다
- 인터럽트가 들어오면 정해진 시간 안에 반드시 처리한다
- 대신 리눅스의 편의는 없다 — 파일도 패키지도 직접 마련해야 한다

잘 맞는 일: 차량 통신 · 주기 제어 · 안전 감시

둘 중 하나를 고르는 문제가 아니다. 예전에는 리눅스 보드 하나와 MCU 하나를 따로 두고 선으로 이었다. 차량용 SoC는 그 둘을 한 칩에 넣고, 사이를 내부 메시지(IPC)로 잇는다. 부품 수 · 배선 · 지연이 함께 줄어든다.

라즈베리파이와 무엇이 다른가

성능 경쟁이 아니다. 자동차가 요구하는 것을 처음부터 갖고 있느냐의 차이다.

	라즈베리파이 4 B · 5	TOPST D3-G
SoC	BCM2711 — Cortex-A72 ×4 @1.8 GHz BCM2712 — Cortex-A76 ×4 @2.4 GHz (파이 5)	TCC8050 — Cortex-A72 ×4 + A53 ×4 + R5
실시간 코어	없다 — 리눅스 하나뿐	**Cortex-R5 + 실시간 OS 내장**
CAN	기본 제공 없음 — 별도 HAT를 얹어야 한다	**3채널 내장** (트랜시버는 보조 보드)
동작 온도	0 ~ +50 °C	−10 ~ +55 °C
용도 등급	소비자 · 교육 · 산업용. 차량 인증(AEC-Q100) 대상이 아니다	차량 전장용으로 개발된 SoC 제품군
40핀 헤더	원조	핀 배열 호환 — HAT와 예제를 그대로 쓴다
자료의 양	**압도적으로 많다.** 검색하면 대부분 나온다	TOPST 공식 문서 저장소 중심. 없으면 물어야 한다

정확히 구분할 것. "차량용 SoC를 얹은 개발 보드"이지 "차량에 장착 가능한 부품"이 아니다. 칩의 등급과 보드의 등급은 다르다 — 이 보드 자체의 동작 온도는 −10 ~ +55 °C다. 실차에 다는 것은 또 다른 문제다.

왜 차량용 반도체로 배우나

편한 길을 두고 굳이 이 보드를 쓰는 이유는 네 가지다.

- **구조를 배우게 된다.** 이기종 코어, CAN, 부팅 체인 — 라즈베리파이에는 없는 것들이다. 자동차 소프트웨어의 어려움은 대부분 이 구조에서 나온다.
- **산업이 이 방향으로 간다.** 존·중앙집중 아키텍처의 '중앙 컴퓨트' 자리에 들어가는 것이 이런 종류의 칩이다. 앞 장의 세 번째 그림을 다시 떠올릴 것.
- **국내에서 실제로 만나게 되는 칩이다.** 텔레칩스는 국산 차량용 반도체 회사이고, TOPST는 그 공식 개발 플랫폼이다. 자동차소프트웨어 분야에서 일하면 이 계열의 칩을 다루게 된다.
- **학기 뒤로 이어진다.** 같은 플랫폼으로 SEA:ME 해커톤, 캡스톤디자인, 자동차비전(4-1) · 자율주행(4-2)까지 연결된다. 여러분의 GitHub 저장소가 그대로 남는다.

> 대신 각오할 것 하나 — **검색으로 안 나오는 문제를 만나게 된다.** 라즈베리파이는 누군가 이미 겪은 문제지만, 이 보드는 아닐 수 있다. 그때 공식 문서를 읽고, 로그를 보고, 물어보는 것 — 그것이 실무에서 하는 일이다.

QUESTION 02

카메라 영상 처리와 모터 제어, 각각 어느 코어에 두겠나?

Cortex-A72(리눅스)와 Cortex-R5(실시간 OS) 중에서 고르고, **왜 그런지**를 말한다.
이어서 한 가지 더 — 만약 둘 다 A72에 둔다면 무엇이 문제가 되겠나?

CORTEX-A72 · A53
리눅스 · 라이브러리 풍부 · 응답 시간 보장 없음

CORTEX-R5
실시간 OS · 응답 시간 예측 가능 · 편의 없음

D

개발 환경

> PC에서 짜서 보드에서 돌린다.
> 그 사이를 잇는 도구가 리눅스 · SSH · ROS2 · Git이다.

두 대의 컴퓨터로 일한다 — 호스트와 타깃

보드에는 모니터도 키보드도 붙이지 않는다. 네트워크로 들어가서 작업한다.



학기 내내 하게 될 일 — **왼쪽에서 코드를 짜고, 오른쪽에서 돌리고, 결과를 다시 왼쪽에서 본다.**

왜 리눅스인가

취향이 아니다. 이 네 가지가 없으면 학기 안에 차를 굴릴 수 없다.

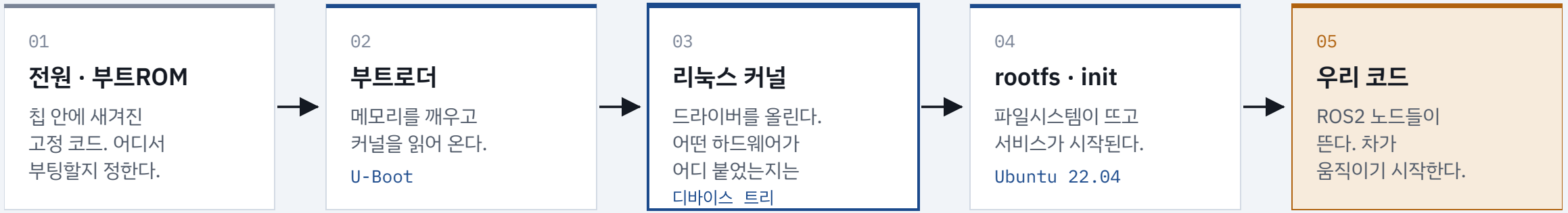
- **드라이버가 이미 있다.** 카메라는 V4L2, 라이다는 USB 시리얼, 센서는 I2C·SPI — 커널이 표준 인터페이스로 이미 제공한다. 우리는 `/dev/video0`을 열기만 하면 된다.
- **도구와 패키지가 있다.** `apt` 한 줄로 Python · OpenCV · GStreamer가 들어온다. 이것을 직접 만들면 한 학기가 그대로 사라진다.
- **ROS2가 리눅스 위에서 산다.** ROS2 Humble의 기준 환경이 Ubuntu 22.04다. 우리가 22.04를 쓰는 이유가 이것이다.
- **원격으로 들어갈 수 있다.** SSH로 접속하고, 로그를 보고, 파일을 주고받는다. 차 위의 보드를 책상에서 고친다.

> 대가도 분명하다 — **리눅스는 언제 끝날지 약속하지 않는다.** 파트 C에서 본 그대로다. 그래서 차량용 SoC는 리눅스 옆에 실시간 코어를 하나 더 둔다. 편의와 보장을 동시에 갖는 방법은 지금까지 이것뿐이다.

학기 내내 여러분이 가장 오래 보고 있을 화면은 **터미널**이다. `cd` `ls` `apt` `ssh` `vim`, 파일 권한 — 이 정도는 손에 붙어 있어야 한다.

보드에 전원을 넣으면 무슨 일이 일어나나

PC의 리눅스와 다른 점은 여기다. 부팅의 각 단계를 우리가 직접 얹는다.

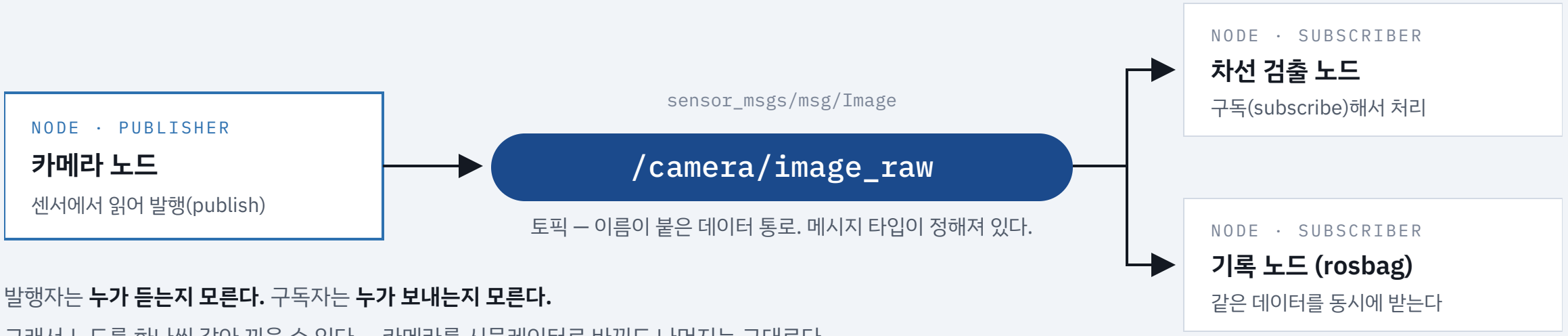


PC는 이 과정을 제조사가 이미 해 두었다. 임베디드 보드는 **우리가 직접 얹는다.** 그래서 안 켜지는 이유도 다섯 군데가 될 수 있다.

부팅 로그를 읽을 줄 알면 어디서 멈췄는지 보인다 — **2강에서 직접 굽고, 부팅시키고, SSH로 들어간다.**

ROS2를 한 장으로 — 노드와 토픽

프로그램을 하나로 짜지 않는다. 작은 프로그램(노드)들이 이름 붙은 통로(토픽)로 대화한다.



발행자는 **누가 듣는지 모른다.** 구독자는 **누가 보내는지 모른다.**

그래서 노드를 하나씩 갈아 끼울 수 있다 — 카메라를 시뮬레이터로 바꿔도 나머지는 그대로다.

왜 하나의 프로그램으로 짜지 않나

센서가 여섯 개, 사람이 두 명이다. 그 조건에서 나오는 답이 노드 분리다.

분리

하나가 죽어도 나머지는 산다

노드는 각각 별개의 프로세스다. 라이다 드라이버가 죽어도 카메라 주행은 계속된다. 그리고 **누구 코드가 문제인지 바로 보인다.**

분업

두 사람이 같은 파일을 건드리지 않는다

A는 `/camera/image_raw`를 내놓고, B는 `/scan`을 내놓는다. 약속한 것은 **토픽 이름과 메시지 타입**뿐이다.

도구

보고 · 기록하고 · 되돌린다

`rviz2`로 라이다 점군을 눈으로 보고, `rosbag`으로 주행을 통째로 녹화해 책상에서 다시 돌린다.

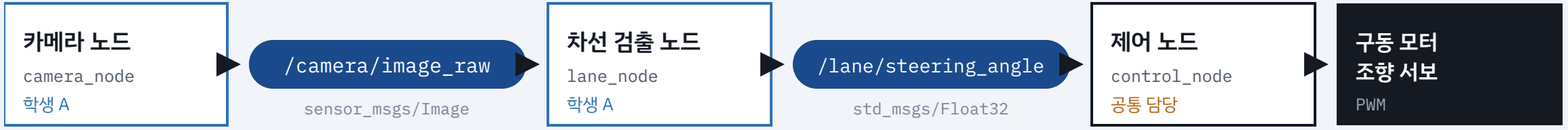
rosbag이 이 과목에서 특히 중요하다. 센서 문제는 대부분 "그때 그 순간"에만 나타난다. 주행을 녹화해 두면 실패를 몇 번이든 다시 재생하며 분석할 수 있다. **녹화하지 않은 실패는 분석할 수 없다** — 랩노트의 증거도 여기서 나온다.

이 과목의 목표는 ROS2 전문가가 되는 것이 아니다. **노드 · 토픽 · 발행 · 구독** 네 개념으로 센서 데이터를 흘려보낼 수 있으면 충분하다.

우리가 만들 시스템 — 노드와 토픽 그래프

6강까지 실선을 완성한다. 점선은 학기가 지나면서 하나씩 붙는다.

지금 만드는 것 — 3 ~ 6강



나중에 붙는 것 — 점선



그리고 /gps/fix , /imu/data

12강 · 학생 B

제어 노드는 **고치지 않는다.**
토픽을 하나 더 구독할 뿐이다.

매주 커밋한다 — 그 저장소가 결과물이다

랩노트는 평가 20%지만, 학기 뒤에 남는 것은 이것 하나다.

저장소 구조 (예시)

```
lab-notebook/
├── README.md
├── notes/
│   ├── week01.md
│   ├── week02.md
│   └── ...
├── src/        ROS2 패키지
└── data/       측정값 · 로그 · 사진
```

저장소 하나에 **기록과 코드를 함께** 둔다. 갈라 두면 둘 다 관리되지 않는다.

랩노트 한 편의 구성

1 이론 요약	이번 주 배운 것 3~5줄. **이해 안 된 것을 반드시 명시**
2 진행 기록	한 것 / 막힌 것 / 시도한 해결책 / 다음 주 계획
3 증거	측정 데이터, 그래프, 스크린샷, 오실로스코프 캡처, **실패 로그**
4 소스 코드	같은 저장소에 커밋

분량이 아니라 **구체성**으로 평가한다. "잘 안 됐다"가 아니라 "3.3 V로 넣었더니 에코 핀이 안 뜨길래 5 V로 바꿔 봤다"가 기록이다.

> 채용·대학원·해커톤에서 "무엇을 만들었는가"를 증명하는 것은 성적표가 아니라 **이 저장소**다. 그리고 커밋 날짜는 나중에 만들어 낼 수 없다 — 15주 동안 매주 찍힌 기록이 그 자체로 증거가 된다.

E

정리 · 다음 주

> 다음 주엔 보드를 켠다.

오늘 한 장으로

다섯 문장만 남기면 된다.

파트	주제	남길 것
A	**우리가 만들 것**	차를 먼저 굴리고 센서를 하나씩 얹는다. 센서마다 **원리 → 연결 → 기능화 → 기록** 4단계를 반복한다.
B	**자율주행 개념**	자율주행은 **인지–판단–제어 루프**다. 센서는 물리량만 준다. 모든 항목에서 이기는 센서가 없어서 여러 개를 겹친다.
C	**임베디드 · 차량 컴퓨팅**	임베디드의 조건은 성능이 아니라 **마감 시각**이다. 차량용 SoC는 리눅스 코어와 실시간 코어를 한 칩에 둔다.
D	**개발 환경**	**PC에서 짜서 보드에서 돌린다.** 노드가 토픽으로 대화하므로 두 사람이 각자 만들어도 붙는다.
E	**태도**	하드웨어 프로젝트는 절반이 실패한다. **실패를 기록한 사람이 이 과목에서 가장 좋은 점수를 받는다.**

오늘 다룬 것 중 **이해가 안 된 항목**을 랩노트 #1의 "이론 요약"에 그대로 적을 것. 모르는 것을 적어 오는 것이 이 과목에서는 감점이 아니라 자료다.

2강 — 보드를 켠다

이론에서 하드웨어로 넘어가는 회차다. 오늘 세운 개발 PC를 그대로 쓴다.

강의 90분

- 임베디드 리눅스 구조 — 부트로더 · 커널 · rootfs · **디바이스 트리**
- 호스트–타깃 개발 흐름 (크로스 개발, 이미지 굽기, 시리얼 콘솔)
- ROS2 아키텍처 — 노드 · 토픽 · 메시지 · **런치 파일**
- 센서 인터페이스 개관 — GPIO · I2C · UART · SPI · MIPI-CSI

실습 90분

- D3-G 부팅 이미지 굽기 (microSD)
- 첫 부팅 · 부팅 로그 함께 읽기
- 네트워크 설정 → **SSH 접속**
- ROS2 설치 확인 → 노드 두 개로 `talker / listener`

2강 제출물 — ① 역할 확정 (A 비전 / B 측거·측위) ② 시스템 아키텍처 초안: 두 사람이 합의한 **노드 이름 · 토픽 이름 · 메시지 타입** 목록

준비물 — 노트북(오늘 셋업한 Ubuntu 환경) · microSD 카드 리더 · 랜 케이블(있으면). 보드와 microSD는 학과에서 준비한다.

랩노트 #1 — 다음 주 수업 전까지

세 가지를 `notes/week01.md` 에 적어 커밋한다. 길 필요 없다.

- **오늘 셋업한 내용.** Ubuntu를 어떤 방식(네이티브 / WSL2 / VM)으로 올렸고 **왜 그것을 골랐는지**. 설치 중 막힌 것과 어떻게 넘었는지. 스크린샷 한 장이면 충분하다.
- **"안개 낀 밤, 어떤 센서가 살아남나?"에 대한 본인의 답.** 센서 하나를 고르고, **근거를 물리 원리로** 쓴다. 수업에서 말한 것과 생각이 바뀌었으면 그 이유도 함께.
- **역할 희망 1문단.** A(인지·비전)인지 B(측거·측위)인지, 왜 그쪽인지. 둘 다 하고 싶으면 그렇게 써도 된다. 2강에 최종 확정한다.

제출 방법

본인 GitHub 저장소에 커밋 → **저장소 URL을 교수에게 한 번만** 전달. 이후는 매주 커밋만 하면 된다.

평가 기준

분량이 아니라 **구체성**. 막힌 것을 그대로 적은 랩노트가 매끄럽게 정리된 것보다 좋은 점수를 받는다.

실습 90분 — 개발 PC를 세운다

오늘 끝내야 다음 주에 보드를 켤 수 있다. 막히면 그 자리에서 같이 뚫는다.

시간	할 일	끝났을 때의 상태
00–10	**환경 점검**	각자 노트북의 OS · 여유 저장 공간 · 관리자 권한 확인. 세 가지 방식 중 무엇을 고를지 결정
10–45	**Ubuntu 22.04 설치**	터미널이 열리고 `lsb_release -a` 가 22.04를 출력한다
45–70	**GitHub · SSH 키**	랩노트 저장소가 생겼고, `ssh -T git@github.com` 이 인증에 성공한다
70–85	**역할 논의**	A(인지·비전) / B(측거·측위) 희망을 서로 말하고, 겹치면 조정 방향을 잡는다
85–90	**마무리**	랩노트 #1 항목 확인 · 2강 준비물 확인

설치가 오늘 안 끝나도 괜찮다. **다만 어디서 멈췄는지를 랩노트에 적어 올 것.** 오류 메시지를 그대로 복사해 두면 다음 주에 바로 이어서 뚫을 수 있다. 화면을 찍어 두는 습관을 지금부터 들인다.

Ubuntu 22.04 — 세 가지 중 하나를 고른다

ROS2 Humble의 기준 환경이 22.04다. 버전을 임의로 올리지 말 것.

방식	장점	단점	이런 사람에게
네이티브 설치 듀얼 부팅	성능 온전. USB · 카메라 접근이 자유롭다	파티션 작업이 필요하고 되돌리기가 번거롭다	리눅스를 계속 쓸 생각이면
WSL2 Windows 11 / 10	설치가 가장 쉽고 Windows와 그대로 병행	USB 장치를 붙이려면 `usbipd-win` 별도 설정. GUI 도구에 제약	**가장 무난한 출발점**
가상머신 VirtualBox 등	안전하고 스냅샷으로 되돌릴 수 있다	느리고 메모리를 많이 쓴다	노트북 사양에 여유가 있으면

SSH 키 만들고 등록하기

```
ssh-keygen -t ed25519 -C "학번@yu.ac.kr"
cat ~/.ssh/id_ed25519.pub
# 출력을 GitHub → Settings →
#   SSH and GPG keys → New SSH key
ssh -T git@github.com   # 인증 확인
```

랩노트 저장소 만들기

```
# GitHub에서 lab-notebook 생성 (Public)
git clone git@github.com:<계정>/lab-notebook.git
cd lab-notebook && mkdir -p notes src data
git add . && git commit -m "init"
git push
```

공개(Public)로 만들 것 — 포트폴리오로 쓰려면 공개여야 한다. 비밀번호·개인정보는 절대 커밋하지 않는다.